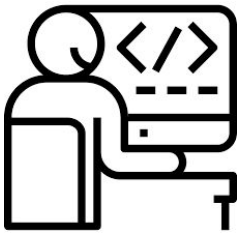


Algoritmos y Programación I

AyPI – **Temas de la clase pasada**

- Cálculo de máximos y mínimos

AyPI – **Temas de la clase de hoy**

- Funciones
- Alcance de variables

AyPI – **MODULARIZACIÓN**

FUNCIONES

FUNCIÓN

Conjunto de instrucciones que realizan una tarea específica y retorna un único valor.

```
Function nombre(...): tipo;
var ...
begin
 ...
 nombre:= valor a retornar;
end;
```

¿Qué diferencias se ven con los procedimientos?

AyPI – **MODULARIZACIÓN**

FUNCIONES

INVOCACIÓN POR SU NOMBRE

Invocación usando variable

El resultado se asigna a una variable del mismo tipo que devuelve la función.

```
program uno;
Function auxiliar(...): real;
Var ...

begin
    ...
    auxiliar:= ...;
end;
Var
  aux:real;
begin
    aux:= auxiliar(...);
    write (aux);
end.
```

El retorno de la función es a la misma línea de invocación

AyPI – **MODULARIZACIÓN**

FUNCIONES

INVOCACIÓN POR SU NOMBRE

Invocación en un while/if

El resultado se asigna a una variable del mismo tipo que devuelve la función.

```
program uno;
Function auxiliar(...): real;
Var ...

begin
    ...
    auxiliar:= ...;
end;
Var
  aux:real;
begin
    while (auxiliar(...) = 5.5) do
        begin
        end;
    if (auxiliar(...) = 5.5) then
        begin
        end;
end.
```


El retorno de la función es a la misma línea de invocación

AyPI – MODULARIZACIÓN

FUNCIONES

Realizar un programa que lea valores numéricos hasta ingresar un valor negativo. Por cada número se debe imprimir el factorial de dicho número.

```
Program Factoriales;
function factorial(n: integer): integer; begin ... end;
var
  valor: integer;
begin
  read (valor);
  while (valor >= 0) do
    begin
    writeln('El factorial de ', valor, ' es ', factorial(valor));
    read(valor);
    end;
end.
```


AyPI – MODULARIZACIÓN

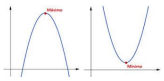
FUNCIONES

Realizar un programa que lea valores numéricos hasta ingresar un valor negativo. Por cada número se debe imprimir el factorial de dicho número.


```
function factorial(n: integer): integer;
var
  i, res: integer;
begin
res:= 1;

for i:= 1 to n do
    res:= res * i;

factorial:= res;
end;
```

AyPI – MODULARIZACIÓN

FUNCIONES



Realizar un programa que lea valores numéricos hasta ingresar un valor múltiplo de N. El programa previamente debe leer el valor de N e informar la cantidad de valores procesados.

```
Program Multiplos;
function esMultiplo(v, m: integer): boolean; begin ... end;
var   N, valor, cont: integer;
begin
  cont:= 0;
  read (N);  read (valor);
  while (not esMultiplo(valor, N)) do
    begin
    cont:= cont + 1;
    read(valor);
    end;
write ('Se procesaron ', cont, ' valores.');
end.
```

AyPI – MODULARIZACIÓN

COMUNICACIÓN

Opción 1

```
function esMultiplo (v, n:integer): boolean;
Var  ok:boolean;
Begin
   if (v MOD n = 0) then ok:= true
   else ok:= false;
   esMultiplo:= ok;
end;
```

Opción 2

```
function esMultiplo(v, n:integer): boolean;
Var  ok:Boolean;
Begin
   ok:= (v MOD n = 0);
   esMultiplo:= ok;
end;
```

Opción 3

```
function esMultiplo(v, n:integer):boolean;
Begin
    esMultiplo:= (v MOD n = 0);
end;
```

AyPI – **MODULARIZACIÓN**

EJERCICIO - 1

```
Program Ej1;
function esNumPrimo (num:integer):boolean;
    begin
    ...
    end;
Var
 encontrados, numero, valor, cuantos:integer;

Begin
   encontrados:= 0;  numero:= 1;
   read(cuantos);
   while (encontrados < cuantos) do
      begin
      read(valor);
      if (esNumPrimo(valor)) then
          begin
          writeln(valor, ‘ es primo.’);
          encontrados:= encontrados + 1;
          end;
      numero:= numero + 1;
      end;
   write(numero);
End.
```

¿Qué hace este programa?

AyPI – **MODULARIZACIÓN**

EJERCICIO - 2

```
Program Ej2;
function esNumPrimo (num:integer):boolean;
   begin
   ...
   end;
Var
 numero, inferior, superior:integer;

Begin
   read(inferior);
   read(superior);
   for numero:= inferior to superior do
      begin
      if (esNumPrimo(numero)) then
          writeln(numero, ‘ es primo.’);
      end;
End.
```

¿Qué hace este programa?

AyPI – **MODULARIZACIÓN**

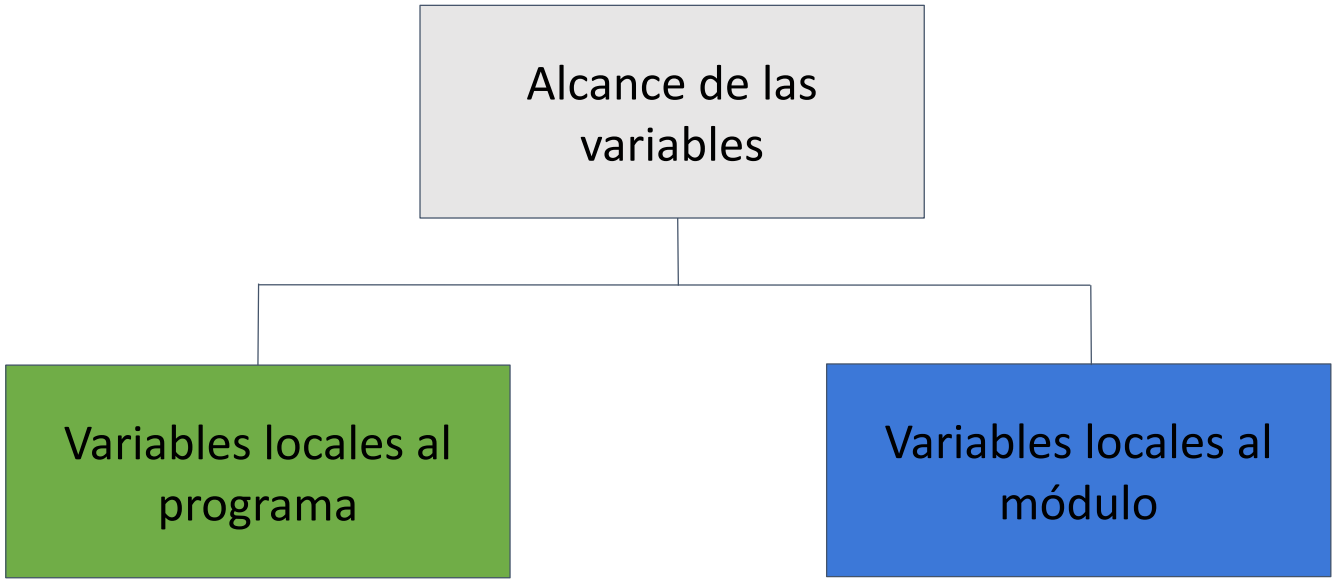
EJERCICIO - 3

```
Program Ej3;

function suma (a, b, c, d: integer): integer;
Begin
  suma:= a + b + c + d;
End;

function promedio(a, b, c, d: integer): real;
begin
  promedio:= suma(a, b, c, d) / 4;
end;

Var
 a,b,c,d:integer;
Begin
  read(a);  read(b);  read(c);  read(d);
  write ( promedio(a, b, c, d) );

  read(a);  read(b);  read(c);  read(d);
  write ( suma(a, b, c, d) );
End.
```

¿Qué hace este programa?

AyPI – **MODULARIZACIÓN**

ALCANCE DE LAS VARIABLES



AyPI – **MODULARIZACIÓN** ALCANCE DE LAS VARIABLES

- Si es una variable utilizada en un módulo
  - Tiene acceso a las variables locales declaradas en el propio módulo
  - Tiene acceso a sus parámetros

- Si es una variable usada en un programa
  - Tiene acceso a las variables locales declaradas en el programa

AyPI – **MODULARIZACIÓN** ALCANCE DE LAS VARIABLES

```
Program alcance;

procedure prueba;
Var
  c: integer;
Begin
End;

Var
  d:integer;

Begin
End.
```

c es una variable local del módulo → Solo pueden ser usadas en el proceso que están declaradas.

d es una variable local del programa → Solo pueden ser usadas en el programa principal

AyPI – **MODULARIZACIÓN**

ALCANCE DE LAS VARIABLES

```
Program alcance;
Const
   ...

Procedure prueba;
Var
  c: integer;
Begin
End.

Var
  d:integer;
Begin
End.
```

¿Dónde pueden ser usadas las constantes?

```
Program alcance;
Const
...

Procedure prueba;
Var
  c: integer;
Begin
End.

Var
  d:integer;
Begin
End.
```

Tanto en el programa principal como en todos los módulos declarados.

AyPI – **MODULARIZACIÓN**

ALCANCE DE LAS VARIABLES

¿Qué imprime?

```
Program alcance;

Procedure prueba;
Var
  x:integer;
Begin
   x:= 34 DIV 3;
   write (x);
End;
Var
  x:integer;
Begin
   x:= 8;
   prueba;
   write (x);
End.
```

Variables del proceso prueba

x:= 11 Imprime 11

Variables del programa (locales)

x:= 8 Imprime 8

AyPI – **Actividad**

Implemente una función que reciba las dimensiones de un terreno (frente y fondo) y retorne el área correspondiente.

Implemente una función que reciba el área de un terreno, y el valor del metro cuadrado en pesos y retorne la valuación fiscal en pesos.

Implemente una función que reciba la valuación fiscal en pesos de un terreno y una tasa de cambio a dólares y retorne la valuación fiscal en dólares.

Implemente un programa que procese la información de los 15 terrenos de una inmobiliaria. El programa debe leer el valor del metro cuadrado en pesos y la tasa de cambio a dólares. Luego, para cada terreno se debe leer los valores del frente y fondo. El programa debe imprimir el área y la valuación fiscal en dólares.